

Guía Práctica: administración y seguridad en PostgreSQL

Resumen teórico de repaso sobre la evolución del control de acceso relacionado a la práctica que realizamos en la clase:

1. Control de Acceso Discrecional (DAC - Discretionary Access Control)

Es el modelo más básico donde el dueño del objeto tiene la autoridad para decidir quién puede acceder a él. Antes esto se gestionaba principalmente mediante permisos directos (GRANT/REVOKE) sobre los usuarios.

Esto tiene una limitación: Cuando un sistema crece (muchos usuarios, muchas tablas), el dueño del objeto debe gestionar manualmente cada permiso para cada usuario o grupo, lo que genera una "matriz de acceso" gigantesca y rígida. Esto es propenso a errores humanos (olvidar revocar permisos al salir alguien).

2. Control de Acceso Basado en Roles (RBAC - Role-Based Access Control)

Este modelo surge precisamente para solucionar la rigidez del DAC, introduciendo el concepto de "función laboral". En lugar de preguntar "¿Qué puede hacer Juan?", se pregunta: "¿Qué rol tiene asignado Juan?", y luego, "¿Qué hace ese rol?".

- Entidades Clave:
 - Usuario: El individuo físico.
 - Rol: La abstracción de la función laboral (ej: rol_vendedor, rol_administrador).
 - Permisos: Los derechos específicos sobre objetos (tablas, columnas, etc.).
 - Sesión: La activación temporal del contexto al ejecutar una tarea.
- Ventaja Clave: Escalabilidad y Principio de Mínimo Privilegio. Si un nuevo vendedor entra, se le asigna rol_vendedor. Si ese rol ya tiene los permisos necesarios (y solo esos), no hay riesgo de sobreprivilegiación por error humano.

En PostgreSQL todo es un ROLE. La diferencia fundamental está en si ese rol debe ser capaz de iniciar sesión directamente:

- `CREATE USER nombre WITH PASSWORD 'pw'` : Es un ROLE que automáticamente tiene el atributo LOGIN. Al ejecutarlo, esa persona puede conectarse al servidor.

- `CREATE ROLE nombre;` : Es un mero rol o grupo de permisos. No tiene LOGIN por defecto y debe ser asignado a un usuario para que este pueda usarlo (o se le añade el atributo LOGIN posteriormente).

Nota: Si creas un ROLE y luego le asocias un USER, estás usando la arquitectura más limpia. El LOGIN es lo que permite la conexión inicial al sistema.

Comenzamos

1. Perfiles de Usuario y Privilegios de Columna

En primer lugar se deben definir los roles que existirán y los permisos que tendrán. Vamos a crear un perfil para el personal de Depósito. Ellos solo deben poder ver los productos y únicamente modificar la cantidad disponible (stock), no el precio.

-- 1. Preparación de roles y usuarios

```
CREATE ROLE rol_deposito;
```

```
CREATE USER deposito_jose WITH PASSWORD 'Tup2026!';
```

```
GRANT rol_deposito TO deposito_jose;
```

```
CREATE ROLE rol_vendedor;
```

```
CREATE USER vendedor_maria WITH PASSWORD 'Tup2026!';
```

```
GRANT rol_vendedor TO vendedor_maria;
```

-- 2. Asignación granular

```
GRANT SELECT ON articulos TO rol_deposito;
```

```
GRANT UPDATE (stock) ON articulos TO rol_deposito;
```

Responde: Si ejecutamos `GRANT UPDATE ON articulos TO rol_deposito;` (sin especificar columna), ¿estaríamos violando el Principio de Privilegio Mínimo? ¿Por qué?

Respuesta: Sí, estaríamos cometiendo una falla grave al darle más permisos de los necesarios. El empleado de depósito solo necesita su labor operativa (modificar stock). Si le damos UPDATE general, a nivel de base de datos le estamos permitiendo modificar el precio_unitario o la descripción, lo cual es una vulnerabilidad de Integridad y excede estrictamente sus funciones.

Prueba de validación:

```
SET ROLE deposito_jose;
```

-- Intento 1: Leer stock, actualizarlo y volver a leer (lo que tiene permitido)

```
SELECT codigo, descripcion, stock FROM articulos WHERE id_articulo = 1;
```

```
UPDATE articulos SET stock = 50 WHERE id_articulo = 1;
```

```
SELECT codigo, descripcion, stock FROM articulos WHERE id_articulo = 1;
```

-- Intento 2: Intentar bajar el precio (lo que NO tiene permitido)

```
UPDATE articulos SET precio_unitario = 1.00 WHERE id_articulo = 1;
```

```
RESET ROLE;
```

Responde: El segundo UPDATE falló. ¿El error lo dio la base de datos por una regla de negocio (restricción CHECK) o por un control de acceso lógico?

Respuesta: Es un control de acceso lógico (Seguridad). El motor SGBD bloqueó la transacción inmediatamente en la fase de Autorización (revisión de privilegios de DAC/RBAC), ni siquiera llegó a analizar si el precio cumplía las restricciones de dominio o los constraints (CHECK precio_unitario > 0).

2. Vistas como Filtros de Seguridad

El equipo de marketing/ventas realizará una campaña exclusiva para clientes "VIP". Necesitan los correos electrónicos de los clientes que tengan un saldo superior a \$100.000 en su billetera virtual.

Por políticas de confidencialidad financiera, los vendedores no deben ver el saldo real de los clientes, ni sus nombres o provincias, solo la lista de emails filtrada.

-- 1. Crear la vista uniendo clientes y billetera, filtrando y seleccionando solo el email

```
CREATE VIEW v_emails_campania_vip AS
```

```
SELECT c.email
```

```
FROM clientes c
```

```
JOIN billetera_clientes b ON c.id_cliente = b.id_cliente
```

```
WHERE b.saldo > 100000;
```

-- 2. Asignar permiso a la vista, NO a las tablas base

```
GRANT SELECT ON v_emails_campania_vip TO rol_vendedor;
```

Responde: Si el usuario rol_vendedor NO tiene permisos de lectura (SELECT) sobre la tabla clientes ni sobre la tabla billetera_clientes, ¿cómo es que la base de datos le permite obtener esos emails a través de la vista?

Respuesta: Porque las vistas, por defecto, se ejecutan con los privilegios del "Propietario" de la vista (quien la creó, usualmente el DBA), no con los del usuario que la ejecuta. El SGBD verifica que el vendedor tenga permiso de leer la vista v_emails_campania_vip, y luego utiliza los permisos de administrador del DBA para acceder a las tablas base, hacer el JOIN y traer el resultado.

Este es un excelente mecanismo de seguridad: logramos darles la información exacta que necesitan para trabajar, sin exponer la información patrimonial (el saldo) ni otros datos personales.

Prueba de validación:

```
SET ROLE vendedor_maria;
```

```
-- Intento 1: Consultar la vista (lo que tiene permitido)
```

```
SELECT * FROM v_emails_campania_vip; -- FUNCIONA: Devuelve solo una  
columna con emails.
```

```
-- Intento 2: Intentar husmear los saldos directamente (lo que NO  
tiene permitido)
```

```
SELECT * FROM billetera_clientes; -- FALLA: permiso denegado.
```

```
RESET ROLE;
```

3. Criptografía en la Base de Datos (Simétrico vs Hash)

Vamos a crear una tabla para medios de pago. El número de tarjeta debe poder recuperarse para procesar el pago (Cifrado Simétrico), pero el CVV (código de seguridad) nunca debe ser visto por nadie, solo validado (Hash) en el momento de confirmar una compra.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
CREATE TABLE medios_pago (  
    id_medio_pago SERIAL PRIMARY KEY,
```

```
id_cliente BIGINT REFERENCES clientes(id_cliente),  
tarjeta_cifrada BYTEA,  
cvv_hash TEXT  
);  
  
-- Insertamos un dato  
  
INSERT INTO medios_pago (id_cliente, tarjeta_cifrada, cvv_hash)  
VALUES (1,  
        pgp_sym_encrypt('4540-1234-5678-9012', 'mi_clave_secreta'),  
        crypt('123', gen_salt('bf'))  
);
```

Responde:

1. Si un hacker roba el archivo .mdf (o el disco) de la base de datos pero no tiene la frase 'mi_clave_secreta', ¿puede leer los números de tarjeta?
2. Si alguien consigue acceso a la base de datos y ejecuta `SELECT * FROM medios_pago`; ¿podrá recuperar información sensible?
3. ¿Por qué usamos `crypt` para el CVV en lugar de `pgp_sym_encrypt`? ¿Qué diferencia hay en la "recuperabilidad" del dato?

Respuestas:

1. No. El número está encriptado y almacenado como bytes ilegibles (BYTEA). El algoritmo simétrico (AES) requiere la clave para descifrar. Sin la clave, el dato robado es basura matemática. A esto apunta la "Confidencialidad".
 2. Tampoco. Al ejecutar ese `SELECT`, el intruso solo verá una cadena de bytes hexadecimal incomprensible en la columna `tarjeta_cifrada` y un texto largo (el hash con su salt) en `cvv_hash`. Para obtener el número real de la tarjeta, necesitaría además conocer la contraseña (`mi_clave_secreta`), la cual idealmente vive en las variables de entorno de la aplicación web, no dentro de la base de datos.
 3. `pgp_sym_encrypt` es bidireccional (puedo volver a ver el número si tengo la clave). En cambio, `crypt` genera un Hash matemático unidireccional. Nadie, ni siquiera el DBA, puede saber que el CVV original era '123'. Solo se puede saber haciendo un nuevo hash del intento del usuario y comprobando si ambos hashes coinciden. (Esto cumple normativas reales como PCI-DSS).
4. Revocacion de permisos y eliminacion de usuarios

Intentemos borrar al usuario `deposito_jose`.

-- Ejecutar esto fallará si José es dueño de algo o tiene permisos

```
DROP USER deposito_jose;
```

Nota: Para evitar problemas de integridad en el catálogo del SGBD. Si se borra al dueño de una tabla, la tabla quedaría "huérfana" y postgres no permite borrarlos con un simple drop. Si se borra un usuario que propagó privilegios (GRANT OPTION), dejaríamos permisos inconsistentes. PostgreSQL obliga a reasignar dueños o limpiar permisos explícitamente para mantener la matriz de control de acceso limpia.

Orden correcto:

```
REVOKE rol_deposito FROM deposito_jose;
```

```
DROP OWNED BY deposito_jose;
```

```
DROP USER deposito_jose;
```